



Padrão ML Lopes Design

Tecnologia e fluxo de trabalho para software próprio

Autor: ML Lopes Design — Marcio (mlopesdesign@gmail.com) **Versão do documento:** 1.0 · 10 de agosto de 2026
Base de evidência: Salgueiro Gestão V2 — em produção desde julho de 2026, versão 3.7.0, 143 rotas, 37 tabelas, 363 asserções de teste.

Como usar este documento

Este é o parâmetro de todo software da ML Lopes Design daqui em diante. Ele existe para ser entregue a uma inteligência artificial — Claude Code, Codex, Antigravity ou outra — antes de a primeira linha ser escrita, e para ser cobrado depois.

Três formas de uso:

Situação	O que fazer
Projeto novo	Copiar o <code>AGENTS.md</code> que acompanha este documento para a raiz do projeto. Para o Claude Code, o mesmo conteúdo com o nome <code>CLAUDE.md</code> .
Projeto em andamento	Ler a seção 9 (armadilhas) antes de tocar em banco de dados ou em campo de formulário.
Contratando alguém	As seções 2, 3 e 7 são a especificação técnica. A seção 8 é o contrato de conduta.

Uma ressalva importante logo de saída. Este documento descreve o que funcionou e o que custou caro. Ele não é uma promessa de que a mesma stack serve para tudo — a seção 2.4 diz explicitamente quando **não** usá-la. Um padrão que não sabe reconhecer os próprios limites vira a causa do próximo problema.

1. O princípio

Software leve, sem runtime, que o autor consegue ler inteiro e o cliente consegue instalar sozinho.

Tudo neste documento decorre disso. Cada decisão técnica foi tomada contra uma alternativa mais moderna e mais pesada, e a leveza ganhou — não por gosto, mas porque o cliente roda o sistema num computador de loja, sem administrador de sistemas e, muitas vezes, sem internet boa.

Quatro consequências práticas:

1. **Zero etapa de compilação.** O código que está no repositório é o código que roda. Abre-se o arquivo, lê-se, corrige-se.
2. **Zero dependência instalada na máquina do cliente.** Nada de Node, nada de .NET, nada de Java, nada de Visual C++ Redistributable.
3. **Atualização é trocar um arquivo.** Sem instalador, sem desinstalar a versão anterior, sem perder dados.
4. **O autor entende cada peça.** Não há caixa-preta que precise de um especialista externo para ser aberta.

2. A stack

■ 2.1 Uma correção de vocabulário

Não usamos **Java**. Java é uma linguagem que exige uma máquina virtual instalada no computador do cliente (JRE) — exatamente o que este padrão evita.

O que usamos é **JavaScript**, que apesar do nome parecido não tem relação com Java. JavaScript é a linguagem que já existe dentro de todo navegador e dentro do Windows moderno. É isso que torna o executável pequeno: a máquina do cliente **já tem** o motor que roda o nosso código.

A confusão é comum e vale registrar aqui, porque uma IA que receba a instrução "usamos Java" vai produzir um projeto inteiramente diferente — pesado, com compilação e com runtime obrigatório.

■ 2.2 As peças

Peça	O que faz	Versão	Vai para o cliente?
JavaScript (ES2020+)	A linguagem. Módulos nativos <code>import/export</code>	—	Sim
HTML + CSS	A tela, escritos à mão	—	Sim
Neutralino.js	Empacota o app como <code>.exe</code> do Windows	6.3.0	Sim (binário pronto)
WebView2	Desenha a tela. Já vem no Windows 10/11	do sistema	Já está lá
sql.js (build asm.js)	SQLite compilado para JavaScript, roda dentro da janela	—	Sim (<code>src/js/vendor/</code>)
PowerShell	Extensão para rede e impressão — nativo do Windows	do sistema	Já está lá
Node.js	Só no ambiente de desenvolvimento: build, testes, geração de PDF	22+	Não
NSIS	Gera o instalador <code>Setup.exe</code>	—	Não

■ 2.3 O que isso produz, em números reais

Medido no Salgueiro Gestão V2, versão 3.7.0:

Artefato	Tamanho	Observação
<code>Salgueiro Gestao.exe</code>	2,78 MB	O binário do Neutralino, com o ícone injetado
<code>resources.neu</code>	5,98 MB	Todo o sistema: telas, regras, banco inicial, bibliotecas
<code>Salgueiro Gestao Setup.exe</code>	15,3 MB	Instalador completo, com SumatraPDF embutido
Requisitos na máquina	nenhum	Windows 10 ou 11 limpo

Para comparação honesta: a versão anterior deste mesmo sistema usava Electron e passava de 150 MB — e o build quebrava. Um projeto equivalente em React + Electron entrega instalador entre 80 MB e 200 MB e exige uma etapa de compilação que ninguém consegue depurar no computador da loja.

■ 2.4 Quando NÃO usar esta stack

Esta é a seção mais importante do documento, e a razão pela qual ele não pode ser lido como "use Neutralino sempre".

Cenário	Por quê	O que usar
Mais de ~50 mil registros no banco	O sql.js carrega o banco inteiro na memória e regrava o arquivo todo a cada escrita. Passando disso, a gravação fica lenta e o risco de corrupção cresce	PostgreSQL ou MySQL com servidor de verdade
Muitas pessoas escrevendo ao mesmo tempo	Não há controle de concorrência real. O modelo multiterminal descrito na seção 3.5 funciona porque um computador é o dono do banco	Servidor com banco cliente-servidor
Aplicativo para celular (loja de apps)	Neutralino é desktop	Capacitor, ou web responsiva instalável (PWA)
Site público ou SaaS multiempresa	Outra natureza: precisa de servidor, autenticação central, isolamento de dados	Stack web com backend próprio
Plugin de WordPress	Ambiente do WordPress: PHP, hooks, banco do WP	PHP + padrão descrito na seção 10

A regra é honesta: este padrão resolve *aplicativo de gestão instalado num computador, com um dono do dado*. Fora disso, o que se aproveita é a **filosofia** (seções 1, 3, 6, 7, 8) e não a lista de tecnologias.

■ 2.5 O que é proibido — e a razão de cada proibição

Uma proibição sem motivo é obedecida enquanto ninguém pergunta. Estas têm motivo.

Proibido	Motivo
TypeScript	O dono do projeto lê e edita o código diretamente. Nada que precise ser compilado antes de rodar.
React, Vue, Angular, Svelte	Trazem etapa de build, dependências e uma camada que esconde o que está acontecendo na tela. O sistema inteiro do Salgueiro é uma SPA escrita à mão em 1.630 linhas de <code>app.js</code> .
Electron	Quebrou a versão 1 deste sistema e está vetado. Além do tamanho, arrasta um Chromium inteiro.
Webpack, Vite, Rollup, Babel	Nenhuma etapa de build no JavaScript. O arquivo do repositório é o arquivo que roda.
Tailwind ou CSS com compilador	Mesmo motivo. CSS é escrito à mão, com variáveis nativas do navegador.
Dependências npm no código do cliente	Biblioteca de terceiros entra como arquivo em <code>src/js/vendor/</code> , baixada uma vez e versionada no Git. Sem <code>node_modules</code> em produção, sem <code>npm install</code> na máquina do cliente, sem atualização silenciosa quebrando o sistema.

3. Arquitetura

3.1 O desenho em uma passada

```

index.html      ← página única. BOM UTF-8 + <meta charset>
|
├─ js/app.js ————— menu, permissões, roteamento de telas
|   └─ api(canal, payload) ← porta única entre tela e regra
|       └─ Neutralino presente → js/backend/servidor.js (local)
|           └─ senão → fetch('/api') (terminal em rede)
|
├─ js/<tela>.js ————— uma por área do menu
|
└─ js/backend/
    ├─ servidor.js — despacha canal → core/*, aplica permissão por rota
    ├─ db.js ————— wrapper do sql.js + migrações
    ├─ ambiente.js — TUDO que toca o sistema operacional
    └─ core/*.js ——— regra de negócio pura
  
```

3.2 A regra que sustenta tudo

A regra de negócio mora em `core/`, em funções puras que recebem `db` como primeiro parâmetro. Sem DOM, sem `window`, sem Neutralino dentro do `core`.

O motivo é prático, não estético: assim cada regra pode ser testada com Node e SQLite de verdade, sem abrir o programa e sem clicar em nada.

No Salgueiro isso não é teoria — é o que permitiu provar que uma correção funcionava **antes** de ela chegar ao cliente. A versão 3.6.0 foi entregue com 51 asserções escritas para ela; o projeto acumula 363.

Assinatura típica de uma função de `core`:

```

// core/produtos.js
export function salvarProduto(db, p, usuario) { ... }
  
```

Ela recebe o banco, recebe os dados, devolve `{ ok: true, ... }` ou `{ ok: false, erro: '...' }`. Nada mais. Isso é o que torna o teste possível: no teste, `db` é um SQLite de mentira — quer dizer, de verdade, só que num arquivo temporário.

3.3 A porta única

Toda conversa entre tela e regra passa por **uma** função:

```
api('assunto:acao', dados)
```

Em `src/js/app.js:40`, uma constante decide o caminho:

```
const NO_APP = typeof window.Neutralino !== 'undefined' && !window.__TERMINAL_REDE;
```

- **No aplicativo** → chama `servidor.js` direto, dentro da mesma janela. Sem rede, sem latência.
- **Num terminal em rede** (outro computador, pelo navegador) → `fetch('/api')` com token de sessão.

A tela não sabe em qual dos dois está. É o mesmo código.

■ 3.4 Permissão é do backend, não da tela

Esconder o botão não é segurança. Em `servidor.js` existe uma tabela `PERM_ROTA` que declara a permissão exigida por `rota`, verificada em `processar()` (`servidor.js:1053`) antes de a regra rodar:

```
const permNec = PERM_ROTA[canal];
if (permNec && !permissoes.pode(sess.usuario, permNec)) {
  return { ok: false, erro: 'Você não tem permissão para esta ação.' };
}
```

A tela também esconde o que o usuário não pode ver — mas isso é conforto, não trava. O catálogo de permissões fica em `core/permissoes.js`, com perfis-modelo (`admin`, `caixa`, `estoque`) que servem de ponto de partida e podem ser ajustados por usuário.

■ 3.5 Multiterminal, quando faz sentido

Um computador é o dono do banco. Os demais entram pelo navegador, num endereço da rede local. Uma extensão em PowerShell (`extensions/rede/servidor-rede.ps1`) sobe um `TcpListener` na porta 8750 e entrega as mesmas rotas por HTTP.

Duas lições que só aparecem quando isso vai para a loja:

- **A sessão precisa ser isolada por terminal.** Sem isso, o caixa que loga na rede vê o painel do administrador. A correção está em `servidor.js:1078` — a sessão do módulo é trocada temporariamente pela sessão de quem chamou, e devolvida no `finally`.
- **A porta precisa ser liberada no firewall pelo instalador.** Pedir isso ao cliente é garantir que não vai funcionar.

4. Dados: a parte que já custou caro

■ 4.1 Como funciona

O banco é um arquivo SQLite em `%APPDATA%/<Sistema>/dados/`. O `sql.js` carrega esse arquivo inteiro na memória ao abrir o programa e **regrava o arquivo inteiro** a cada escrita, com um atraso de 300 ms para agrupar gravações seguidas (`db.js:72`).

Isso é simples e rápido — e tem três consequências que precisam estar na cabeça de quem escreve qualquer linha que grave dado.

■ 4.2 Dado de alta frequência usa `runVolatil()`, nunca `.run()`

```
// db.js:44 – aplica na memória, NÃO agenda gravação em disco
runVolatil(...params) { return this._exec(params, false); }
```

O chat interno da versão 3.0.0 registrava presença de usuário a cada 4 segundos com `.run()`. Cada ping regravava o banco inteiro. Isso multiplicou por cerca de mil o número de gravações e ajudou a apagar os dados de um cliente em produção.

A regra permanente: **se o dado é descartável e frequente, é `runVolatil()`**.

■ 4.3 Gravação de arquivo nunca pode ter janela de inexistência

A versão que apagou os dados fazia: escreve temporário → **remove o arquivo** → move o temporário por cima. Entre o remove e o move, o banco simplesmente não existia no disco. Se o processo morresse ali — updater reiniciando, queda de energia, fechamento forçado —, acabou.

A sequência correta está em `ambiente.js:64`:

```
1. escreve .tmp          (conteúdo novo, completo)
2. renomeia o atual → .old (nunca apaga – só encosta)
3. move .tmp → arquivo   (existe de novo)
4. remove .old           (só aqui o antigo morre)
```

Em qualquer instante existe pelo menos uma cópia íntegra. E no boot, `_recuperarSeFaltando()` (`ambiente.js:94`) reconstrói o banco a partir do `.old` ou do `.tmp` se o arquivo principal sumiu.

Detalhe do Windows: `filesystem.move` falha se o destino já existir. É por isso que o passo 2 existe — não dá para simplesmente mover por cima.

■ 4.4 Validar backup é olhar o conteúdo, não só abrir

A validação antiga testava `SELECT COUNT(*) FROM usuarios`. Um banco vazio passava — e foi assim que uma restauração "bem-sucedida" deixou o sistema zerado.

O que existe hoje (`db.js:126`) é uma radiografia: conta usuários, produtos, vendas e clientes, marca `tabelas0k` quando alguma tabela essencial está ausente e `temDados` quando há movimento de verdade. O backup diário só grava banco **com dados** — senão, abrir o sistema quebrado transformaria o estrago no backup do dia.

■ 4.5 Migração de banco: o que o SQLite não deixa fazer

Acrescentar coluna é fácil (`ALTER TABLE ... ADD COLUMN`, dentro de `try/catch` para ser idempotente).

Alterar um **CHECK** é impossível. Para aceitar um valor novo numa coluna com restrição, a tabela precisa ser reconstruída inteira: cria a nova, copia, confere a contagem, renomeia. Tudo em transação, com rollback deixando o banco intacto.

A armadilha: a guarda que decide se a migração já rodou tem que testar o valor **entre aspas**. Testar `cortesia` sem aspas casa com o nome da coluna `cortesia_valor`, a guarda dá positivo e a migração nunca roda. Isso aconteceu.

Outra: o carregador de schema divide o `schema.sql` por ponto-e-vírgula. Um **;** dentro de comentário parte o **CREATE TABLE** seguinte — o erro aparece como um `syntax error` obscuro e a tabela some sem alarde.

5. Atualização online

É o recurso que mais impressiona o cliente e o que mais exige cuidado, porque roda sem ninguém olhando.

■ 5.1 O ciclo completo

- | | |
|----------------------------|---|
| 1. Desenvolvimento | → bump de versão + build → resources.neu |
| 2. GitHub Releases | → tag vX.Y.Z + o resources.neu anexado |
| 3. App do cliente pergunta | → <code>api.github.com/.../releases/latest</code> |
| 4. Compara versões | → só oferece se a de lá for MAIOR |
| 5. Usuário clica | → BACKUP obrigatório antes de tudo |
| 6. curl.exe baixa | → resources.neu.tmp |
| 7. move /Y | → substitui o arquivo |
| 8. Grava o banco | → antes de reiniciar |
| 9. restartProcess() | → volta já na versão nova |

■ 5.2 Os pontos que não podem faltar

Só oferece versão maior. `updater.js:36` compara semver e exige `> 0`. Isso tem uma consequência que já custou uma versão: **não adianta trocar o arquivo de uma release existente**. Quem já está naquela versão nunca receberá a correção. Se o `.neu` mudou, a versão sobe — inclusive num rebuild do mesmo dia.

Backup antes de qualquer coisa. `configuracoes.js:1210` chama `backup:preAtualizacao` e **aborta a atualização** se o backup falhar. Sem ponto de retorno, não começa.

Gravar o banco antes de reiniciar. `restartProcess()` mata o processo. Se houvesse escrita pendente no atraso de 300 ms, ela se perderia — e se o processo morresse dentro de uma gravação, o arquivo ficaria incompleto. Por isso `backup:salvarAgora` é chamado imediatamente antes (`configuracoes.js:1246`).

Baixar com `curl.exe`, não com `fetch`. O WebView2 bloqueia o download por CORS. O `curl.exe` é nativo do Windows 10+ e resolve (`configuracoes.js:1229`).

■ 5.3 O texto da release — formato fixo

Todo build termina com estes dados prontos, sem o autor precisar pedir:

```
Tag:      vX.Y.Z
Título:  vX.Y.Z – descrição curta

## O que mudou
- item 1
- item 2

## Como atualizar
1. Baixe o resources.neu abaixo
2. Substitua em %LOCALAPPDATA%\<Sistema>\resources.neu
3. Reinicie o aplicativo

Ou aguarde a notificação automática dentro do app.

## Arquivo
- resources.neu – SHA256: <hash>
```

■ 5.4 O que nunca muda

Estes valores são a identidade do sistema instalado. Mudar qualquer um quebra a atualização automática e o banco de todos os clientes:

- `applicationId`
- `binaryName`
- caminho da pasta de dados (`%APPDATA%\<Sistema>\dados`)
- nome do arquivo do banco
- dono e nome do repositório no GitHub

6. Versionamento

Fonte de verdade única: o campo `"version"` do `neutralino.config.json`.

Tipo de alteração	Sobe	Exemplo
Correção de bug	patch	3.5.1 → 3.5.2
Funcionalidade nova	minor	3.6.0 → 3.7.0
Quebra de compatibilidade	major	2.9.0 → 3.0.0

Qualquer alteração que gere um build novo exige bump. Sem exceção, inclusive rebuild do mesmo dia (ver 5.2).

A versão aparece em três lugares e os três têm que bater:

1. `neutralino.config.json` → `"version"`
2. `src/js/app.js` → `let APP_VERSION = (fallback)`
3. `src/js/backend/ambiente.js` → `versaoApp()` (fallback)

Os dois últimos são só rede de segurança: o valor real vem do `NL_APPVERSION`, que o Neutralino injeta a partir do config.

7. O fluxo de trabalho

■ 7.1 Git

Um commit ao fim de cada sessão de trabalho, com mensagem que explica a causa raiz, não só o que mudou:

```
v3.5.2 – fix: estoque inicial do produto novo era descartado pelo filtro da tela
```

Branch única `main`, remote no GitHub. Repositório só local não é backup.

Fora do repositório: `node_modules/`, `dist/`, `*.log`, binários que o build regenera, e — isto importa — **material bruto do cliente**. Fotos, vídeos e `.psd` estouram qualquer repositório. No Salgueiro são 2,2 GB que ficam em backup próprio, nunca no Git.

Dentro do repositório, apesar de binário: o `resources.neu` de entrega e o banco inicial do cliente. São artefatos de entrega, não subprodutos de build.

■ 7.2 A documentação entra na mesma entrega

Não é etapa opcional nem "depois". Toda alteração estrutural produz, junto:

1. `docs/MANUAL-DO-USUARIO.md` — completo, escrito **para leigo**: o que é, para que serve, passo a passo numerado, o que acontece por trás, e uma linha na tabela de problemas comuns.
2. `docs/GUIA-RAPIDO.md` — o resumido, "em uma folha". Só o dia a dia.
3. **Os dois PDFs regerados** — é o que vai para a loja.
4. `GRAPHIFY.md` — o mapa técnico, regerado por script (`node tools/graphify.js`). Nunca editado à mão.

5. **Bloco novo no topo do arquivo de novidades** — o "o que mudou" que o próprio usuário lê dentro do sistema.

6. **Entrada nova no histórico de versões** do arquivo de instrução.

Tom dos manuais: frase curta, sem jargão, tabela quando houver comparação. Explicar **por que** existe, não só onde clicar.

■ 7.3 O GRAPHIFY

Um script lê o código e gera um mapa técnico: rotas com a permissão de cada uma, tabelas, módulos, os pontos que mexem em estoque, as invariantes e as armadilhas conhecidas.

Serve para duas coisas:

- **A IA lê antes de mexer em qualquer coisa estrutural.** Em vez de vasculhar 3.400 linhas, ela vê o mapa.
- **O autor confere o impacto** de uma mudança antes de aprová-la.

Regra: **regerar a cada alteração estrutural**, nunca editar à mão.

■ 7.4 Testes

Não existe framework. Existe um arquivo `.mjs` que roda com Node, abre um SQLite de verdade, executa a regra e conta asserções. O resultado é um número que vai na mensagem de entrega:

"Verificação: 51 asserções — 30 no chat, 21 no vale impresso. Total do projeto: 363."

O que testar, em ordem de importância:

1. **O caminho mínimo, não só o completo.** O bug da versão 3.5.2 morava exatamente no campo que ninguém preenche: quem cadastra peça sem variação deixava cor e tamanho em branco, e a linha nunca saía da tela. Os testes anteriores sempre preenchiam a grade inteira — por isso passou.
2. **A migração sobre um banco no formato antigo.** Não sobre um banco novo.
3. **A regra que mudou**, contra SQLite de verdade.
4. **A tela**, com jsdom, disparando eventos reais — quando não for possível abrir o programa.

E uma autocrítica que vale como regra: **reportar um bug inexistente custa tão caro quanto não achar o real**. Na versão 3.6.0 um teste mal escrito acusou "conversa direta quebrada"; o defeito era do teste. Antes de reportar, confirme que o teste fala a mesma língua do código.

■ 7.5 Antes de entregar — o checklist

1. `node --check` em todo arquivo tocado
2. Cada comando do `schema.sql` roda isolado
3. Teste automatizado da regra que mudou
4. Teste da migração sobre banco antigo
5. Nenhum erro no console
6. Versão batendo nos três lugares

7. Documentação e GRAPHIFY na mesma entrega
8. Dados da release prontos, no formato da seção 5.3

8. Como conduzir uma IA

Esta seção é o contrato de conduta. Ela vale para qualquer agente e para qualquer projeto.

8.1 Diagnóstico

Nunca teorizar, supor ou "achar". Todo diagnóstico cita arquivo e linha.

Faltou informação para concluir? **Pergunte**. Não preencha a lacuna com hipótese.

Um diagnóstico errado entregue com confiança custou os dados de um cliente em produção. Essa regra nasceu daí, e é a que mais importa.

Na prática, a diferença entre as duas frases abaixo é a diferença entre o padrão e o prejuízo:

Inaceitável	Correto
"Provavelmente o filtro está descartando a linha."	"app.js:1279 filtra com <code>linhas.filter(l => (l.cor l.tamanho l.id))</code> — quem cadastra sem variação deixa os três vazios, então a linha nunca sai da tela."

8.2 Entrega

- **O autor não executa `.bat`, comando de terminal nem passo manual.** Entregue pronto. Só peça algo quando for impossível fazer sozinho.
- **Sem downgrade. Sem versão intermediária quebrada.**
- **Arquivo completo**, nunca trecho solto para colar.
- **Interface 100% em português**, escrita para quem não é técnico.
- **Explicações simples.** O destinatário é designer e desenvolvedor, não precisa de jargão para entender o que quebrou.

8.3 O que uma boa entrega contém

Olhando um registro real de versão do Sagueiro, o formato é sempre o mesmo:

1. **O sintoma**, na voz de quem reportou — *"preencho, salvo e fica zerado"*
2. **A causa raiz**, com arquivo e linha — e, quando forem várias somadas, todas
3. **A correção**, explicada
4. **Por que passou despercebido** até agora

5. A **verificação**, com número de asserções

6. A **lição**, quando houver

O item 4 é o que impede a repetição. O item 6 é o que faz o histórico valer mais que o código.

■ 8.4 O histórico de versões é parte do sistema

O arquivo de instrução guarda, por versão, a causa raiz e a correção. Não é changelog de marketing — é o que impede o próximo agente (ou o próximo mês) de repetir um erro já pago.

Quando uma entrega descobre uma armadilha nova, ela entra ali **escrita como aviso**, em maiúsculas se for grave:

ARMADILHA: o `blur` de um campo dispara antes de o foco chegar no próximo.

9. Catálogo de armadilhas já pagas

Cada uma destas custou pelo menos uma sessão de trabalho. Algumas custaram dados.

■ Banco e arquivos

Armadilha	Regra
Escrita frequente com <code>.run()</code> regravava o banco inteiro	Dado descartável e frequente usa <code>runVolatil()</code>
Remover antes de mover deixa o arquivo inexistente	<code>tmp</code> → renomeia atual para <code>.old</code> → move → apaga <code>.old</code>
<code>filesystem.move</code> falha no Windows se o destino existir	Por isso o <code>.old</code>
SQLite não altera <code>CHECK</code>	Reconstruir a tabela, em transação, conferindo a contagem
Guarda de migração sem aspas casa com nome de coluna	Testar o valor entre aspas
<code>;</code> dentro de comentário no <code>schema.sql</code> parte o <code>CREATE TABLE</code> seguinte	Rodar cada comando isolado antes de entregar
Validar backup só contando uma tabela deixa banco vazio passar	Radiografar todas as tabelas essenciais e exigir movimento

■ Interface

Armadilha	Regra
O <code>blur</code> dispara antes de o foco chegar no próximo elemento. Redesenhar a tela no handler destrói o elemento que o usuário acabou de clicar	Recalcular só o necessário; nunca redesenhar a região vizinha no <code>blur</code>
Campo numérico que escuta <code>change</code> e redesenha mata o cursor — não dá para digitar	Escutar <code>input</code> , aceitar vazio durante a digitação, normalizar no <code>blur</code> /Enter
Listener em fase de captura com <code>stopPropagation()</code> sequestra o clique de elementos internos	Excluir explicitamente as áreas que precisam do clique
Sem <code><meta name="viewport"></code> , o celular renderiza como desktop de 980 px	Incluir sempre, mesmo em app desktop
Campo com fonte menor que 16 px faz o iOS dar zoom sozinho ao focar	<code>font-size: 16px</code> nos inputs
Aviso só por som falha em máquina sem som	Todo aviso precisa de forma visual persistente

■ Windows e encoding

Armadilha	Regra
Sem BOM UTF-8 no <code>index.html</code> , o WebView2 quebra todos os acentos	BOM (EF BB BF) e <code><meta charset></code> . Ao editar o arquivo por script, reescrever o BOM à mão
<code>fetch</code> para download externo é bloqueado por CORS no WebView2	<code>curl.exe</code> via <code>execCommand</code>
Ícone da barra de tarefas some após atualizar	Ícone personalizado na pasta de dados, que o updater nunca toca
Nome de impressora com espaço quebra o comando	Passar o arquivo como primeiro argumento e usar splatting no PowerShell

■ Processo

Armadilha	Regra
Trocar o arquivo de uma release publicada não atualiza ninguém	O updater só oferece versão maior . Se o build mudou, a versão sobe
Testar só o caminho completo esconde o bug do caminho mínimo	Testar o formulário preenchido pela metade
Teste mal escrito reporta bug inexistente	Confirmar que o teste usa o mesmo formato de dado que a tela envia
Compressão LZMA no instalador estoura o tempo e gera arquivo truncado	Usar zlib: 8 segundos em vez de mais de 60

10. Adaptando a outros tipos de projeto

O que **não** muda, seja qual for a plataforma:

- Regra de negócio separada da tela, testável isoladamente
- Uma porta única entre interface e regra
- Permissão verificada no backend, não na tela
- Versão com fonte de verdade única e bump obrigatório
- Documentação e mapa técnico na mesma entrega
- Diagnóstico com arquivo e linha
- Histórico de versões com causa raiz

■ Plugin de WordPress

Equivalência	No desktop	No WordPress
Fonte de verdade da versão	<code>neutralino.config.json</code>	Cabeçalho do arquivo principal do plugin
Regra de negócio pura	<code>src/js/backend/core/</code>	<code>includes/</code> — classes sem <code>echo</code> , sem HTML
Porta única	<code>api(canal, payload)</code>	AJAX/REST com <code>nonce</code> e <code>current_user_can()</code>
Migração de banco	<code>migrar(db)</code> no boot	Versão do schema em <code>option</code> , comparada no <code>plugins_loaded</code>
Atualização online	GitHub Releases + <code>resources.neu</code>	Updater próprio apontando para GitHub Releases, ZIP com estrutura preservada
Identidade imutável	<code>applicationId</code> , <code>binaryName</code>	Slug, pasta raiz e arquivo principal — nunca mudam

Regras específicas que já estão consolidadas: preservar slug, pasta raiz, arquivo principal, updater e estrutura do ZIP; nunca entregar trecho solto; empacotar e validar o ZIP antes da release.

■ SaaS ou sistema web

A stack muda (precisa de servidor e banco cliente-servidor), mas a separação continua: regra pura no servidor, uma porta de API, permissão no backend, migração versionada, e o mesmo rito de versão e documentação.

11. Começando um projeto novo — o dia 1

1. Criar a pasta **em disco local**. Nunca em unidade de rede ou pasta sincronizada de nuvem: o ambiente de build não monta esses caminhos, e o projeto trava.
2. Copiar o `AGENTS.md` deste padrão para a raiz (e a cópia `CLAUDE.md`, se for usar o Claude Code).
3. Definir a **identidade imutável**: `applicationId`, `binaryName`, pasta de dados, nome do banco. Escrever isso no `AGENTS.md` antes de escrever código.
4. Criar o repositório no GitHub e configurar o remote. Primeiro commit ainda vazio.
5. Escrever o `schema.sql` antes das telas.
6. Primeiro módulo de core com o primeiro teste. Antes da primeira tela.
7. Só então a interface.

■ Ordem que funciona

```
schema.sql → core/ + testes → servidor.js (rotas) → telas → build → manual
```

Escrever a tela primeiro parece mais rápido e não é: a regra acaba dentro do handler do botão, deixa de ser testável, e o primeiro bug em produção não terá como ser reproduzido.

Resumo em uma página

Assunto	A regra
Linguagem	JavaScript puro. Não Java, não TypeScript
Empacotamento	Neutralino 6 + WebView2. Nunca Electron
Banco	SQLite via sql.js, arquivo em <code>%APPDATA%</code>
Framework	Nenhum. Nem front, nem build, nem CSS compilado
Bibliotecas	Arquivo em <code>vendor/</code> , versionado. Sem npm no cliente
Arquitetura	Regra pura em <code>core/</code> , uma porta <code>api()</code> , permissão no backend
Alta frequência	<code>runVolatil()</code> , nunca <code>.run()</code>
Gravação	<code>tmp</code> → <code>.old</code> → move → apaga. Nunca remover antes
Versão	Fonte única, bump a cada build, três lugares batendo
Atualização	GitHub Releases. Só versão maior. Backup antes, gravar antes de reiniciar
Git	Commit por sessão, remote obrigatório, sem material bruto
Documentação	Manual + guia + PDFs + GRAPHIFY, na mesma entrega
Teste	Node contra SQLite real. Caminho mínimo, migração antiga
Diagnóstico	Arquivo e linha. Nunca supor
Entrega	Pronta, completa, em português, sem passo manual
Limite	Até ~50 mil registros e um dono do dado. Acima disso, outra arquitetura

ML Lopes Design · Marcio · mlopesdesign@gmail.com Documento vivo: cada armadilha nova entra na seção 9 na mesma entrega em que foi descoberta.